Lineage Is Memory: Build on the Best Parent and Let It Accumulate What Feedback Reveals

Shanglin Yang, Zhan Shi, Li Yin

Abstract

LLM-guided optimizers such as GEPA improve a prompt by showing a proposer model a few failing examples and asking it to rewrite. Much recent work tries to make that step better. It shows the proposer more of the error distribution through full-set clustering, targeted minibatches or agents that read every trace, or it pays for a stronger proposer. We show that a simpler choice matters more: whether each rewrite builds on the current best candidate (lineage) or starts again from the seed.

In a controlled grid at equal task-call budget, building on the best parent adds +0.238 held-out accuracy on seven task families with gpt-4.1-mini and +0.222 with claude-haiku-5-5. This is larger than any change to what the proposer is shown. Switching the proposer from gpt-4.1-mini to gpt-4.1 adds nothing (−0.019 [−0.053, +0.014]), while lineage still adds +0.380 under the stronger proposer. Lineage also absorbs most of what wider reflection buys. A full-failure summary that keeps the expected answer is the best display when rewriting the seed (+0.156 over the first four failures) and ties raw failures under lineage (−0.018 [−0.053, +0.010]). The same pattern holds on Haiku.

The mechanism is memory. A child inherits the rules its parent already learned, so facts revealed by feedback accumulate across rounds instead of having to fit in one prompt. On the Banking77 intent benchmark, the reference loop at first lost to GEPA by 0.29. Logs traced the gap to one sentence in GEPA's reflection prompt that asks the proposer to copy task facts from the feedback into the instruction. Adding a sentence of that kind, a fix designed after the diagnosis and pre-registered before running, raises our loop from 0.372 to 0.856. That is above GEPA's 0.664 by +0.192 [+0.097, +0.289], at 2.3× fewer tokens. With lineage, the best prompts name 11–12 of the 12 intents; rewriting the seed with the same sentence they name 4, and score 0.317. On BIG-Bench Hard error detection, where there is no vocabulary to carry, the fix does not help (−0.061), and the methods tie.

Every pre-registered test that failed is reported in §4.8. We release the harness.

1 Introduction

Reflective prompt optimizers have become the default way to tune LLM programs. GEPA (Agrawal et al., 2025) runs the current program on a small minibatch, shows a reflection model the inputs, outputs and feedback, and accepts the rewrite if it scores higher; Pareto-front parent selection keeps the search building on what already works. The same loop, with scores instead of text feedback, drives OPRO (Yang et al., 2023), EvoPrompt (Guo et al., 2023) and the program-evolution systems behind FunSearch (Romera-Paredes et al., 2023) and AlphaEvolve (Novikov et al., 2025).

Most attempts to improve these loops target the reflection step. ESPO (Liu et al., 2026) argues from a coupon-collector bound that a minibatch of three failures covers a small fraction of the error modes per round, and replaces it with full-set error clustering. RPT (Bayat et al., 2026) gives the optimizer a tool that clusters critiques over the whole training set. AdaGEPA (Chen et al., 2026) swaps minibatch slots toward identified weaknesses. CASD (Singh et al., 2026) lets a coding agent read the full trace corpus. The common alternative is to pay for a stronger proposer. All of these were evaluated inside lineage-based search, where each candidate rewrites the current best. None turns lineage off, so none can say how much of the loop's progress comes from what the proposer sees and how much from what it inherits.

We treat an LLM-guided optimizer as a loop with three knobs. **P** (lineage) is which candidate the proposer rewrites. **F** (disclosure) is what it is shown about failures. **A** (scheduling) is how candidates are evaluated and stopped. We cross P and F in a single controlled grid at equal task-call budget. We measure coverage per

failure category rather than only aggregate score, and we decompose cost by call type. The tasks are seven families: five constructed so that the needed rules can only be learned from feedback, a free-text task scored by an LLM judge, and ScoNe (She et al., 2023). We add two public benchmarks, Banking77 (Casanueva et al., 2020) and BIG-Bench Hard salient-translation-error detection (Suzgun et al., 2022). We replicate on a second model family and test a stronger proposer.

Our findings:

1. **Lineage is the main lever, and it is free.** Building on the best parent adds +0.238 [+0.224, +0.252] with gpt-4.1-mini and +0.222 [+0.204, +0.239] with claude-haiku-5-5. On the public benchmarks it adds +0.088 [+0.045, +0.130]. With gpt-4.1 as proposer, it adds +0.380 [+0.337, +0.425], while the stronger proposer itself adds nothing over gpt-4.1-mini (§4.1–4.2).
2. **Lineage is memory, and it can be fed.** On Banking77, the facts the task model needs (the intent names) are visible only in feedback. With lineage, an instruction to carry those facts into the prompt makes them accumulate: the best prompts name 11–12 of 12 intents, against 4 when every rewrite starts from the seed. Held-out accuracy rises from 0.372 to 0.856, above GEPA's 0.664 at 2.3× fewer tokens (§4.3). The fix was designed after a diagnosis of where GEPA won; it was pre-registered before it ran and is tested on two tasks.
3. **Lineage makes better failure samples largely redundant.** A diverse sample and a full-failure summary that keeps the expected answer both help when rewriting the seed and add nothing under lineage. The interactions are +0.068 and +0.174 on gpt-4.1-mini, and +0.122 and +0.187 on Haiku. GEPA-style random minibatches are worse than raw first-k failures under lineage (−0.061 [−0.086, −0.035]). Without lineage, partial disclosure can even push coverage of unshown failures below showing nothing (§4.4–4.6).
4. **Most of GEPA's extra cost is validation, not reflection.** At matched task calls, building on the best parent with early stopping needs 2.6–3.3× fewer tokens to solve than GEPA where both solve. Reflection is 2–15% of GEPA's tokens; on the public benchmarks it is 8–12% (§4.7).

The practical upshot is a default and a reporting rule. The default: build on the best parent, show raw failures with their expected answers, and tell the proposer to keep what the feedback reveals. The rule: a method that changes what the optimizer sees should be evaluated with lineage switched off as well as on, or its gains cannot be attributed.

2 Related work

LLM-guided program evolution. FunSearch (Romera-Paredes et al., 2023) and AlphaEvolve (Novikov et al., 2025) have a proposer model rewrite programs drawn from an archive of high scorers. Islands, archives or Pareto fronts decide which parent is rewritten next; ImprovEvolve (Kravatskiy et al., 2026) continues this line. In this family the gain is attributed to evolution itself, meaning lineage plus selection, and what the evaluator tells the proposer is treated as an engineering detail. The closest methodological study, Evolution or Illusion? (Oved et al., 2026), sweeps how a fixed proposal budget is split between seeds and iterations and finds that strategy rankings flip with budget. It does not vary what the proposer sees, does not count tokens, and treats best-of-N only as a reference line. We hold the budget fixed and vary the two factors it leaves alone: lineage and disclosure.

Reflective prompt optimization. ProTeGi (Pryzant et al., 2023) and TextGrad (Yuksekgonul et al., 2024) turn failing examples into textual "gradients"; OPRO (Yang et al., 2023) shows the optimizer only a trajectory of scores. GEPA (Agrawal et al., 2025) combines reflection on a random minibatch of three examples with Pareto parent selection, and it has become the default baseline. Its reflection prompt asks the model to "identify all niche and domain specific factual information about the task and include it in the instruction"; §4.3 shows that this sentence, not the minibatch or the Pareto front, accounts for GEPA's advantage over our loop on Banking77. Later analyses question what drives GEPA. (Lumer et al., 2026) find that the *form* of feedback matters: per-dimension feedback transfers, a scalar does not. They attribute validation-to-held-out gaps to a winner's curse under noisy judges. (Giummarra et al., 2026) report that GEPA helps on MATH-500 but not on HotpotQA with local models, and that it raises token use. (Ma et al.,

2024) argue that LLM optimizers rely mostly on their own prior rather than on the error feedback they receive, which matches our null on ScoNe and value labels (§4.5).

Wider reflection scope (2026). A cluster of 2026 papers argues that a few random failures under-cover the error distribution. ESPO (Liu et al., 2026) states the problem as incomplete error observation. It bounds coverage with a coupon-collector argument (about 15 rounds for a 95% chance of seeing every mode at minibatch three) and replaces the minibatch with full-set error clustering into 3–7 patterns, followed by Diversify and Stabilize steps. RPT (Bayat et al., 2026) gives the optimizer a diagnosis tool that clusters critiques over the whole training set. AdaGEPA (Chen et al., 2026) swaps at most one minibatch example per round to target an identified weakness. CASD (Singh et al., 2026) lets a coding agent read the full corpus and reports wins over GEPA on three of four benchmarks at lower cost [abstract-level]. Contrastive Reflection (Koh et al., 2026) selects failures by decision-tree slices, a +1.4-point gain that lies inside its bootstrap interval. Each of these is evaluated inside lineage-based search, usually GEPA itself. None measures per-category coverage against a no-feedback prior, and none turns lineage off. We find that the problem they address is real when the proposer rewrites a fixed seed, and that lineage removes most of it (§4.4, §4.6).

Search efficiency. Explore More, Validate Less (Zhou and Cheng, 2026) observes that GEPA spends evaluations on all-correct minibatches that yield no edit, and proposes fewer full validations. It does not break GEPA's budget down by call type; §4.7 does, and shows that reflection is a small share.

Representative examples. Choosing a small set of examples that represents a larger one is an old problem in program synthesis (Pu et al., 2017) and in-context learning. Our *diverse* arm borrows that idea; what is new here is measuring its effect inside an evolution loop and against lineage.

3 Setup: the P/F/A loop

Loop. Each run starts from a fixed seed: a one-line instruction for prompt tasks, or a deliberately naive function for code tasks. It spends a fixed budget of candidate evaluations, 8 for prompts and 12 for code. In each step a proposer model receives the objective, one candidate and the evaluator's feedback on it, and returns a rewrite, which is scored on the search set. The best search-set candidate of the run is then scored once on a disjoint held-out set, which is never used for selection. A run stops early when a candidate scores 1.0 on the search set.

P (lineage). *seed*: every proposal rewrites the seed, so proposals are independent samples conditioned on the same feedback (best-of-N). *parent*: every proposal rewrites the current best candidate, so fixes accumulate.

F (disclosure). The evaluator shows up to four failing cases with the expected answer. The arms are:

- *none*: score only.
- *first*: the first four failures in dataset order.
- *diverse*: four failures picked by farthest-point selection under edit distance on a coarse shape of the input.
- *minibatch*: four examples drawn at random, right or wrong, as in GEPA's reflective minibatch.
- *cluster*: all failures are passed to one extra model call that groups them into three to seven patterns with one verbatim example each, and only that summary is shown. This is a stand-in for the full-set diagnosis of ESPO (Liu et al., 2026), and its tokens are charged to the run.
- *clustergold*: as *cluster*, but each pattern's example keeps its input and expected answer (§4.6).

A (scheduling). Our loop evaluates every candidate on the full search set and stops at the first perfect score. GEPA 0.0.7 (Agrawal et al., 2025) is run as published, with minibatch 3, Pareto parent selection and the same task-model call budget (8 × search-set size). It is run again with an early-stop hook that ends the run at its first perfect full-set evaluation.

Tasks. There are seven families in the main grid:

- Five constructed families in which the needed rules cannot be read off the task statement: duration and size parsing (code, Python functions run in a sandbox), value labels (a closed label set with idiosyncratic boundaries), date normalisation, and ticket routing.

- One free-text family, support replies, scored by a gpt-4.1-mini judge at temperature 0 against per-example rubrics.
- ScoNe (She et al., 2023), using its six negation conditions as categories (6 search and 12 held-out pairs per condition, label-balanced).

Two further public benchmarks were added in a pre-registered extension:

- **Banking77** (Casanueva et al., 2020): 12 of its 77 intents, 24 search and 36 held-out queries. The seed names no intent labels, so the label vocabulary is visible only in feedback.
- **BIG-Bench Hard salient-translation-error detection** (Suzgun et al., 2022): 18 search and 36 held-out items. The shared instructions and options are moved into the seed, and inputs longer than the evaluator's 300-character feedback window are dropped.

Every example carries a category label, so we measure coverage (the share of candidates that get every example of a category right) in addition to the aggregate score.

Models and runs. In the main grid, gpt-4.1-mini is the task model, the proposer and the clusterer. Each of the 9 cells (seed-none and {seed, parent} × {first, diverse, minibatch, cluster}) is run with 10 seeds per task. The replication uses claude-haiku-5-5 in all three roles (§4.1, §4.4). The strong-proposer arm uses gpt-4.1 as proposer and clusterer with gpt-4.1-mini as task model (§4.2). Intervals are 95% bootstrap intervals over runs; contrasts that average over tasks resample runs within each task. Each protocol and its success threshold were registered before its runs (Appendix A).

4 Results

All numbers are held-out best scores: the mean over 10 seeds with a 95% bootstrap interval. Cross-task contrasts weight tasks equally (`results/final-7tasks.txt`, `results/round4.txt`, `results/round5.txt`). Post-hoc analyses are labelled as such.

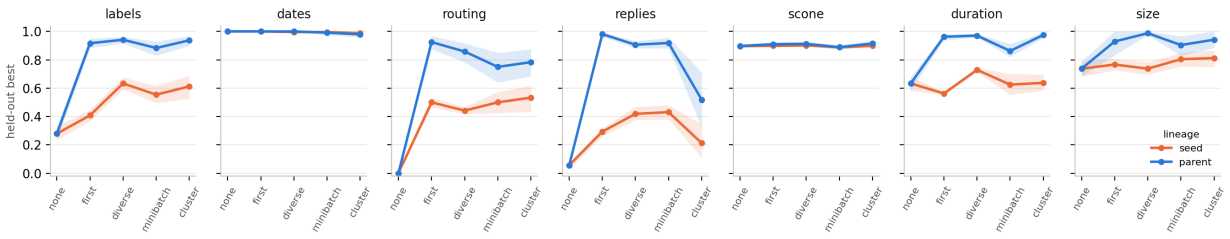


Figure 1: Held-out best by disclosure, without (seed) and with (parent) lineage.

Figure 1. Held-out best by disclosure scheme on the seven main tasks, seed row (orange) versus parent row (blue); bands are 95% intervals over 10 seeds. "none" is shared by both rows.

4.1 Lineage is the main lever

Averaged over the four disclosure schemes and seven tasks, building on the best parent raises the held-out score by +0.238 [+0.224, +0.252]. Among the four pre-registered schemes, no contrast within a row exceeds 0.10. The gold-keeping summary of §4.6 widens the seed-row spread to 0.156, still well below the lineage effect. On the three tasks with room to move and rules that must be learned from feedback (labels, routing, replies), every pre-registered parent cell beats every pre-registered seed cell (Fig. 1). The one crossing involves the sensitivity arm, where seed-row clustergold on replies (0.794) exceeds parent-row cluster (0.519). Dates and ScoNe are flat: dates is solved with no feedback at all (seed-none 1.000), and ScoNe sits at 0.89–0.92 in every cell.

Second model family. The effect carries over to claude-haiku-5-5 as task model, proposer and clusterer. On five tasks (labels, routing, replies, duration, size), averaged over three disclosure columns, lineage adds +0.222 [+0.204, +0.239] (pre-registered H1).

Public benchmarks. On Banking77 and BIG-Bench Hard together, averaged over four disclosure columns, lineage adds +0.088 [+0.045, +0.130] (pre-registered P1). The effect comes from Banking77 (+0.158 [+0.077, +0.241]). On BBH it is +0.017 [−0.006, +0.039]: every cell there lies between 0.61 and 0.68, and seed-none already reaches 0.556.

4.2 A stronger proposer does not substitute for lineage

We replaced the proposer and clusterer with gpt-4.1, kept gpt-4.1-mini as task model and judge, and ran five cells (seed-none and {seed, parent} × {first, clustergold}) on labels, routing and replies. All three pre-registered tests hold:

- Lineage adds +0.380 [+0.337, +0.425] (S1); with the mini proposer the same gap is +0.365 [+0.316, +0.413].
- The gold-keeping summary helps far more without lineage: I_clustergold = +0.294 [+0.201, +0.381] (S2).
- Building on the best parent, it is slightly worse than first-k: −0.057 [−0.101, −0.016] (S3).

Cell by cell, the stronger proposer does not beat the mini one (secondary, not pre-registered as a test):

Cell	gpt-4.1 − gpt-4.1-mini proposer
parent-first	−0.019 [−0.053, +0.014]
parent-clustergold	−0.025 [−0.087, +0.057]
seed-clustergold	−0.069 [−0.170, +0.033]
seed-none	+0.002

At these budgets, buying a stronger proposer bought nothing, while choosing to build on the best parent was worth about 0.38.

4.3 Lineage is memory: carrying what feedback reveals

The gap. On Banking77, GEPA with minibatch 3 reached 0.664 held-out at the same task-call budget, against 0.372 for our loop with lineage and first-k failures. The difference is +0.292 [+0.111, +0.464] (+0.289 with early stopping). On BBH the two tied (−0.019 [−0.050, +0.014]). No run of either method solved the search set on either task, so the pre-registered token-ratio test (R4) could not be evaluated there.

Diagnosis (post hoc, from the logs). Two observations explain the gap.

- *Stalling at zero.* The Banking77 seed names no labels and scores 0. First-generation rewrites mostly fixed the output format (snake_case) and still scored 0, so the best parent stayed the seed. In two of ten runs all eight candidates scored 0, and the final prompt was the 36-character seed. Lineage cannot help on a flat plateau.
- *Fact transfer.* The feedback revealed 12 intent names. Our best prompts named [6, 11, 4, 11, 7, 9, 4, 4, 0, 0] of them, which tracks their held-out scores [0.47, 0.69, 0.31, 0.75, 0.39, 0.58, 0.22, 0.31, 0, 0]. GEPA's best prompts named a median of 9. GEPA 0.0.7's reflection prompt explicitly asks the model to copy "niche and domain specific factual information" from the feedback into the instruction; our proposer's objective had no such sentence. BBH has no vocabulary to carry, and there the methods tie.

Fix (pre-registered before running). We appended one sentence to the proposer's objective and changed nothing else: *"The task model sees only this prompt, never the feedback: copy every task-specific fact the feedback reveals (for example the exact expected labels or formats) into the prompt."* We ran seed-first and parent-first with it on both public tasks, 10 seeds each.

task	parent-first	seed-first + carry	parent-first + carry	GEPA mb3
Banking77	0.372	0.317	**0.856**	0.664

task	parent-first	seed-first + carry	parent-first + carry	GEPA mb3
BBH ste	0.653	0.614	0.592	0.633

On Banking77 the fix adds +0.483 [+0.328, +0.644] (F1). It overtakes GEPA by 0.192: GEPA − ours = −0.192 [−0.289, −0.097] (F2), at 45.8k against 106.9k tokens per run (2.3×). No run stalls at zero (0/10, against 2/10 before).

The fix works only with lineage. Rewriting the seed with the same sentence scores 0.317, and lineage adds +0.539 [+0.494, +0.583] on top of it (pooled over both tasks, F4: +0.258 [+0.221, +0.296]). The prompts show why. With lineage, the best prompts name 11–12 of the 12 intents in every run. Rewriting the seed, they name exactly 4 in every run, the intents in the four failures every seed-row round sees. A child keeps the labels its parent already wrote down and adds those its own feedback reveals, so the vocabulary accumulates across rounds. A proposer that restarts from the seed can only write down what one round shows it.

On BBH, the fix does not help. It lowers parent-first by −0.061 [−0.119, −0.006], so the pre-registered no-harm test F3 fails. GEPA and our fixed loop are within the noise there (+0.042 [−0.008, +0.094]). Carrying facts pays when the feedback contains a vocabulary the task model cannot guess, and it is not a general improvement.

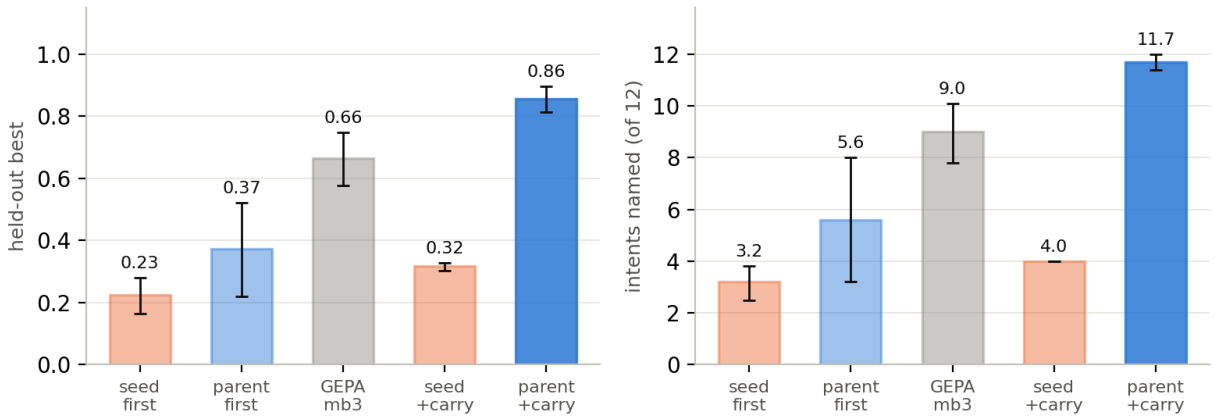


Figure 2: Banking77: carrying facts across lineage.

Figure 2. Banking77 held-out best (left) and number of the 12 intents named in the best prompt (right). Orange: rewrite the seed; blue: build on the best parent; grey: GEPA (minibatch 3, same task-call budget); "+carry" adds the one-sentence fact-carrying instruction. Bars are 95% intervals over 10 seeds (`results/fig2.py`).

4.4 Lineage absorbs most of what a better sample buys (C2)

Without lineage, a diverse sample beats the first four failures by +0.061 [+0.046, +0.077]; with lineage the same contrast is −0.007 [−0.025, +0.012]. The pre-registered interaction is I_diverse = +0.068 [+0.043, +0.092], so C2 is supported and refutation R1 does not hold. On claude-haiku-5-5 the interaction is +0.122 [+0.088, +0.158] (pre-registered Hd): diverse helps rewriting the seed (+0.072 [+0.055, +0.090]) and does not help building on the parent (−0.050 [−0.082, −0.020]). The Haiku first-k cells come from the earlier replication batch, not interleaved with the diverse cells. Routing is the exception on both models: on Haiku its interaction is −0.092 [−0.142, −0.042], because there diverse is worse than first-k without lineage.

The interaction for the clustered summary is larger, +0.120 [+0.073, +0.167], but for a different reason: in the seed row the summary helps a little (+0.038 over first), while in the parent row it is worse (§4.6). With a summary that keeps the expected answer the interaction grows to +0.174 [+0.127, +0.222] (§4.6).

4.5 Without lineage, partial disclosure can crowd out unshown failures (C1)

In the seed row every proposal sees the seed's failures, so the categories that are and are not shown are fixed. On the two code tasks, showing the first four failures lowers the share of proposals that get an unshown category fully right *below* showing none: duration −0.055 [−0.082, −0.033], size −0.113 [−0.144, −0.082]. The intervention on duration confirms the direction: showing only ISO-style failures lowers unshown coverage by −0.048 [−0.079, −0.021], while a random sample of four raises it by +0.061 [−0.003, +0.121]. Value labels show no crowding-out (+0.018 [−0.031, +0.067]), so C1 holds on 2 of 3 constructed tasks, meeting the registered threshold. On ScoNe every seed-row disclosure arm is within ±0.01 of seed-none (R3 holds). Because ScoNe is near ceiling, this is a boundary condition, not evidence of no effect.

4.6 Wider reflection scope helps only without lineage

In the parent row the ordering is first ≈ diverse > minibatch ≈ cluster. GEPA-style random minibatches lose −0.061 [−0.086, −0.035] against first, and the clustered summary loses −0.082 [−0.119, −0.045]. The pre-registered prediction that the clustered summary beats the best alternative in both rows (R2) is refuted in the opposite direction: −0.044 [−0.078, −0.018] in the seed row and −0.095 [−0.132, −0.063] in the parent row. The replies deficit has a visible cause. The clustering prompt asks for a representative wrong answer, and on replies, whose expected outputs are long rubrics, only 31 of 156 summaries kept the expected answer, against 95–100% on every other task.

A summary that keeps the target (post hoc, registered before running). We changed one phrase of the clustering prompt so that each pattern carries a verbatim failure *with its expected answer*. We then reran both rows on all seven tasks (10 seeds, same budget). Every summary kept the expected answer (replies 65 of 65). On replies the parent row recovers from 0.519 to 0.869 (+0.350 [+0.062, +0.612]), so the earlier deficit was information loss, not summarization per se.

Across tasks the picture splits by lineage. Rewriting the seed, the gold-keeping summary is the best arm we ran: +0.156 [+0.122, +0.188] over first-k and +0.095 [+0.058, +0.127] over diverse. Its gains are on labels (0.408→0.683), replies (0.294→0.794) and routing (0.500→0.625). Building on the best parent, it is indistinguishable from first-k (−0.018 [−0.053, +0.010]); the interval excludes an average gain above +0.01, and the contrast is −0.002 without replies. It is also indistinguishable from diverse (−0.011), still beats random minibatches (+0.043 [+0.005, +0.075]), and loses on routing (−0.083 [−0.158, −0.008]). Lineage still adds +0.139 [+0.096, +0.182] on top of the best summary.

The summary is not free: it costs an extra model call per round, and median search tokens per run reach up to 2.2× those of first-k (routing, seed row). Full-set diagnosis as ESPO specifies it is therefore a real fix for the seed-rewriting regime, and redundant once the search builds on its best parent. The claude-haiku-5-5 replication reproduces this split (pre-registered): I_clustergold = +0.187 [+0.152, +0.221] (H2), and under lineage the summary ties first-k at −0.008 [−0.017, +0.002] (H3). Under lineage, random minibatches again lose to first-k (−0.093 [−0.123, −0.067]).

4.7 Where GEPA's tokens go (C3)

At the same task-call budget, evolve with lineage and early stop solves the search set at least as often as GEPA with early stop on four of five tasks: labels 7 vs 6, dates 10 vs 10, routing 9 vs 2, replies 10 vs 2. It solves less often on ScoNe (2 vs 4). Where both solve often enough to compare, GEPA needs 3.34× [2.23, 4.49] (labels) and 2.59× [2.30, 4.84] (dates) the tokens to solve.

Refutation R4 (the gap is explained by GEPA's lack of early stopping) is rejected. Early stopping changes GEPA's median tokens-to-solve by at most 6% on labels, dates and replies; routing has too few solved runs to say. Only on ScoNe does it halve them (97.8k → 50.6k).

Of GEPA's tokens, reflection is 2–15%; full re-evaluations take 21–67% and minibatch checks 27–77%. On the public benchmarks, reflection is 8% (Banking77) and 12% (BBH), and full re-evaluations are 74% and 58%. GEPA used 2.3–2.8× the tokens per run of our loop there.

4.8 Boundary conditions and robustness

These are the results that limit the claims above. All pre-registered tests that failed are listed here.

- **Disclosure contrasts on the public benchmarks (pre-registered P2–P5) do not hold.** The interactions I_clustergold (+0.017 [−0.119, +0.151]) and I_diverse (−0.014 [−0.118, +0.093]) are not distinguishable from 0. The parent-row tie test P3 (+0.021 [−0.101, +0.143]) fails on width: its upper bound exceeds +0.05. The minibatch penalty P5 (−0.011 [−0.113, +0.090]) is not distinguishable from 0. On BBH, every disclosure contrast is within ±0.04. On Banking77, intervals are about ±0.24 because individual runs range from 0 to 0.92. On these two tasks, what the proposer was shown mattered less than lineage did, and no disclosure scheme was reliably better than first-k.
- **Carrying facts can cost a little where there are none to carry.** On BBH the fix lowers parent-first by −0.061 (F3 fails), and under the fix lineage does not help there either (−0.022 [−0.086, +0.039]).
- **Routing** is the one task where diverse samples hurt without lineage (both models), and where clustergold loses under lineage.
- **ScoNe** is near ceiling (0.89–0.92 in every arm), and GEPA solves it more often than our loop (4 vs 2).
- **Leave-one-task-out (post hoc).** Dropping one main-grid task at a time, I_diverse stays positive (+0.046 to +0.094), and parent-row minibatch − first stays negative (−0.042 to −0.069), in all seven subsets. The ungrounded clustered deficit does not: without replies, parent-row cluster − first is −0.019 [−0.045, +0.008]. On Haiku, H1 and H2 stay positive with any one task dropped, and the H3 upper bound stays at or below +0.009.
- **Noise.** Same-seed reruns of five replies-cluster runs differed by up to 0.25, making that cell the noisiest in the grid.

5 Discussion

Why lineage is memory. A proposer that rewrites the seed starts every round from the same failures, so whatever it is shown is the whole of its information about the task. If the sample is skewed, every round inherits the skew, and crowding-out (§4.5) is the visible symptom. A proposer that rewrites the best parent inherits every rule earlier rounds already learned, and the failures it sees next are the residual ones, which shrink and shift as search proceeds. Coverage then accumulates across rounds instead of having to be delivered in one prompt. This is the coupon-collector argument of (Liu et al., 2026) run in the other direction: if each round fixes what it saw and keeps it, the rounds collect the coupons, and the per-round sample does not need to cover the distribution.

The logs agree (post hoc, first-k cells, `results/mechanism_posthoc.txt`). Over a run, the failures shown to a seed-row proposer are always the same four, 17–25% of the search set. The failures shown to a parent-row proposer cover 54% (labels), 61% (routing) and 86% (replies) of it, and a child keeps 70–91% of the examples its parent already got right. Banking77 makes the memory literal (§4.3). With lineage and an instruction to keep what feedback reveals, the prompt becomes a growing list of the intents seen so far, 11–12 of 12 by the end of the budget. Without lineage, the same instruction yields the same 4 intents in every run.

Why this matters more than a stronger proposer. Disclosure and proposer strength both act on a single round. Lineage acts across rounds. At a budget of eight evaluations, a proposer that gets one round slightly better gains less than a loop that keeps every round's gains. That is what we see: gpt-4.1 in place of gpt-4.1-mini changes the best score by −0.019, and lineage changes it by +0.38.

What this means for practitioners. If your optimizer builds on its best candidate, as GEPA, FunSearch-style archives and our loop do, the cheap defaults are these:

- Show raw failures with their expected answers.
- Tell the proposer that the task model never sees the feedback, so anything the feedback reveals must be written into the prompt.
- Spend the budget on more rounds, not on wider reflection or a bigger proposer.
- Do not draw a random minibatch of right and wrong examples: under lineage, it was the one display choice that hurt in every leave-one-task-out subset on gpt-4.1-mini and on Haiku.

Full-set diagnosis pays off when the system cannot build on a parent: parallel best-of-N rewrites of one prompt, one-shot repair from a log of failures, or a first round before any lineage exists. Whatever the format, the expected answer must survive it. A summary that kept what went wrong but not what was wanted lost 0.46 on free-text replies.

What this means for evaluating reflection-scope methods. A reflection-scope fix measured inside GEPA measures its residual value on top of lineage. In our loop that residual is near zero against raw first-k failures but positive against a random minibatch (+0.043 [+0.005, +0.075]). Gains reported over GEPA's default are therefore consistent with our results, and may come largely from replacing the random minibatch, which showing raw failures also does. We did not run ESPO's full pipeline inside GEPA; this is a reading, not a test. Papers proposing such fixes should report a seed-rewriting row (lineage off) next to the usual one, compare against a no-feedback prior, and report per-category coverage of failures the proposer was not shown. Without the seed row, a method that works by substituting for lineage and a method that adds to it look the same. The same holds for comparisons between optimizers: on Banking77, one sentence in the reflection prompt was worth more than GEPA's entire search machinery.

Where the tokens go. GEPA's extra cost is in validation, not in thinking. Reflection is 2–15% of its tokens; the rest is full re-evaluations and minibatch checks, and an early-stop rule does not remove the gap. Efficiency work on reflective optimizers should target the validation schedule first.

6 Limitations

- **The carry-facts fix is post hoc in design.** It was formulated after seeing where GEPA won on Banking77, and pre-registered only before its own runs. It is tested on two tasks, helps on one and slightly hurts on the other. Whether it helps on other vocabulary-bound tasks is untested.
- **Task construction.** Five of the main families are constructed so that the needed rules cannot be inferred from the task statement. On the two public benchmarks the lineage effect is smaller (+0.088), and the disclosure contrasts do not replicate (§4.8).
- **Benchmark preprocessing.** Banking77 uses 12 of 77 intents (20 gave at most 0.15 within 8 evaluations in a smoke run). BBH drops inputs longer than the 300-character feedback window (198 of 250 kept). Both choices were fixed before the pre-registration, and smoke runs are excluded.
- **Single judge.** Free-text replies use one LLM judge (gpt-4.1-mini, temperature 0) with per-example rubrics.
- **Weak proxy for diversity.** "Diverse" uses Levenshtein farthest-point selection, a weak proxy for semantic categories; the clustered arm covers this gap.
- **Two model families, reduced coverage.** The full grid uses gpt-4.1-mini. The claude-haiku-5-5 replication covers five of the seven tasks, with the first-k cells of its diverse arm taken from an earlier batch. The strong-proposer arm covers three tasks. The replies judge is gpt-4.1-mini throughout, and the clusterer is always the proposer's model.
- **IFBench not run.** Its verifier requires executing external code, which was not approved in this study.
- **ScoNe near ceiling.** Held-out is 0.89–0.92 in every arm, so its null result cannot separate no effect from no headroom.
- **Clustered arm.** It implements only ESPO's Diagnose step, not Diversify or Stabilize. Its pre-registered prompt did not require the expected answer; the gold-keeping variant was added after seeing results (registered before running) and is reported as a sensitivity analysis.
- **Batch drift.** The sensitivity, extension and fix arms ran as separate batches; drift in the API models between batches cannot be excluded.
- **Lost credit.** Three batches lost API credit mid-run. Each was quarantined and rerun in full under a rule written before its results were seen (Appendix A); the quarantined runs are archived, not analysed.

7 Pre-registered tests

The pre-registrations were written in Chinese in `09-experiments.md` before each batch of runs, timestamped, and only appended to afterwards; each protocol was also registered with its success threshold in

`experiments.json`. Below is an English translation of the tests. Where the translation and the original differ, the original governs. All intervals are 95% task-stratified bootstrap intervals (10,000 resamples of runs within each cell), and tasks are weighted equally.

A.1 Main grid (2026-10-27; code frozen at git d5ab8923 plus `results/code/`, SHA256SUMS). Model gpt-4.1-mini for task, proposer and clusterer (and for the replies judge, temperature 0). Seeds 0–9. Budget 8 evaluations for prompt tasks and 12 for code tasks. Metric: held-out score of the best search-set candidate; held-out data is never used for selection. Cells: `seed-none` and {seed, parent} × {first, diverse, minibatch, cluster} on seven tasks.

- *C2 (primary).* For X = diverse and X = cluster, I_X = mean over tasks of (X − first)_seed − (X − first)_parent. Supported if the lower bound of I_diverse > 0. R1 refutes if both I_diverse and I_cluster contain 0. R2 (collapse) if cluster − max(other disclosures) has lower bound > 0 in both rows.
- *C1 (crowding out).* In the seed row, coverage of unshown failure kinds, `seed-full-first` vs `seed-none`. Supported if the upper bound of (first − none) < 0 on at least two of labels, duration and size. R3: if every seed-row full-* minus seed-none contains 0 on ScoNe, C1 is narrowed to rules that can only be learned from feedback (ScoNe was flagged as near ceiling before running).
- *C3 (GEPA tokens).* Median tokens-to-solve, GEPA with early stopping vs `parent-full-first`. R4: if the ratio CI contains 1 on at least 3 of 5 tasks, C3 is restated as "the gap comes from missing early stopping".

A.2 Sensitivity arm `sens-clustergold` (2026-10-27, registered after round-1 review and before running; code in `results/code_round1/`). Only the cluster prompt changes: the representative failure now carries input, wrong answer and full expected answer. The pre-registered cluster results are kept and not replaced.

- *S1 (information loss).* On replies, parent clustergold − cluster has lower bound > 0, and at least 80% of the summaries keep the expected answer.
- *S2.* Parent clustergold − first on seven tasks: upper bound < 0 means summaries are harmful even with the target kept; an interval containing 0 means indistinguishable; lower bound > 0 favours full-failure diagnosis.

A.3 Second-model replication `rep-haiku` (2026-10-27). claude-haiku-5-5 as proposer and task model (default adaptive thinking); replies judge unchanged. Tasks: labels, routing, replies, duration, size. Cells: `seed-none`, {seed, parent} × {first, minibatch, clustergold}; n = 10.

- *H1.* Lineage main effect (mean of the three columns), lower bound > 0.
- *H2.* I_clustergold, lower bound > 0.
- *H3.* Parent clustergold − first, upper bound < +0.05.
- Any H that fails is reported as "does not hold on Haiku", without added cells.

A.4 Extension (2026-10-28T16:05 −07:00; code in `results/code_round4/`, data hashes in `DATA_SHA256SUMS`). Design choices fixed after smoke runs and before registration, with smoke runs excluded from analysis: Banking77 reduced from 20 to 12 intents because 20 gave at most 0.15 within 8 evaluations; for BBH salient-translation-error detection, the shared instructions and options were moved into the seed prompt, inputs longer than the 300-character feedback limit were dropped (198 of 250 kept), and the search set was raised from 2 to 3 per class after the 2-per-class smoke run overfit (search 1.0, held-out 0.50). One smoke run showed Banking77 parent clustergold 0.22 vs parent first 0.72 (n = 1); the tests were not changed.

- *`public-bench`.* gpt-4.1-mini on banking77 and bbh_ste, nine cells (`seed-none` and {seed, parent} × {first, diverse, minibatch, clustergold}) run interleaved. P1 lineage (four-column mean), lower bound > 0. P2 I_clustergold, lower bound > 0. P3 parent clustergold − first, upper bound < +0.05. P4 I_diverse, lower bound > 0. P5 parent minibatch − first, upper bound < 0. A task with seed-none ≥ 0.90 is treated as at ceiling and every test is reported with and without it. A failed test is reported as "does not hold on public benchmarks".
- *`gepa-public`.* GEPA 0.0.7 (minibatch 3, with and without early stopping), 10 runs per task, against `parent-full-first`. R4 holds on a task if both arms solve at least 3/10 and the ratio CI contains 1.

- *strong-proposer.* gpt-4.1 as proposer and clusterer, gpt-4.1-mini as task model and judge; labels, routing, replies; cells `seed-none`, {seed, parent} × {first, clustergold}. S1 lineage (mean of the two columns), lower bound > 0; S2 I_clustergold, lower bound > 0; S3 parent clustergold − first, upper bound < +0.05.
- *rep-haiku-diverse.* claude-haiku-5-5, {seed, parent}-full-diverse on the five replication tasks. Hd: I_diverse, lower bound > 0, using the first-k cells from `rep-haiku` (not interleaved).

A.5 Fix arm `fix-carryfacts` (2026-10-28, written in `09-experiments.md` after the diagnosis of §4.3 and before any fix run; code in `results/code_round5/`). One sentence (`CARRY_FACTS`, quoted in §4.3) is appended to the proposer's objective; nothing else changes. gpt-4.1-mini in all roles; banking77 and bbh_ste; cells `seed-full-first` and `parent-full-first`; n = 10; compared against `parent-full-first` and GEPA mb3 from A.4 (not interleaved). Because the fix was designed after seeing where GEPA won, the paper labels it post hoc in design.

- *F1.* Banking77 parent-carry − parent-first, lower bound > 0.
- *F2.* Banking77 GEPA − parent-carry: an upper bound ≤ 0 means the gap is explained by the meta-prompt; otherwise not fully explained.
- *F3 (no harm).* BBH parent-carry − parent-first, lower bound > −0.05.
- *F4.* Lineage under carry-facts (parent-carry − seed-carry, mean of both tasks), lower bound > 0.
- Outcome: F1, F2 and F4 hold; F3 fails (−0.061 [−0.119, −0.006]) and is reported in §4.8.

References

Lakshya A Agrawal, Shangyin Tan, Dilara Soylu, Noah Ziems, Rishi Khare, Krista Opsahl-Ong, Arnav Singhvi, Herumb Shandilya, Michael James Ryan, Meng Jiang, Christopher Potts, and Koushik Sen. Gepa: Reflective prompt evolution can outperform reinforcement learning. arXiv (Cornell University), 2025. URL http://arxiv.org/abs/2507.19457.

Farima Fatahi Bayat, Moin Aminnaseri, Pouya Pezeshkpour, and Estevam Hruschka. Reflective prompt tuning through language model function-calling. arXiv preprint, 2026. URL http://arxiv.org/abs/2605.21781v1.

Iñigo Casanueva, Tadas Temčinas, Daniela Gerz, Matthew Henderson, and Ivan Vulić. Efficient intent detection with dual sentence encoders. arXiv preprint, 2020. URL http://arxiv.org/abs/2003.04807v1.

Junyang Chen, Zecheng Wang, and Jingbang Chen. Adagepa: Adaptive feedback allocation for reflective prompt optimization. arXiv (Cornell University), 2026. URL https://arxiv.org/abs/2609.39927.

Andrea Giummarra, Carmelo Baglieri, Antonio Di Maria, Andrea Farruggio, Michele Piatto, and Giovanni Micale. Understanding when reflective prompt and skill optimization helps on locally served reasoning models: A two-model study of gepa and skillopt. arXiv (Cornell University), 2026. URL https://arxiv.org/abs/2610.26020.

Qingyan Guo, Rui Wang, Junliang Guo, Bei Li, Kai-Tao Song, Xu Tan, Guoqing Liu, Jiang Bian, and Yujiu Yang. Evoprompt: Connecting llms with evolutionary algorithms yields powerful prompt optimizers, 2023. URL https://www.semanticscholar.org/paper/a9c75cf664f675a1b4034b0256ec3c5168e293df.

Derek Koh, Jinghui Mo, Benjamin H. Le, Jiening Zhan, Baofen Zheng, Kevin Bevis, Nathaniel C. Owen, Lauren Elizabeth Charney, Wenqiong Liu, and Jingwei Wu. Contrastive reflection for iterative prompt optimization. arXiv preprint, 2026. URL http://arxiv.org/abs/2606.30840v1.

Alexey Kravatskiy, Valentin Khrulkov, and Ivan Oseledets. Improvevolve: Basin-hopping meets llm-guided evolutionary search. arXiv (Cornell University), 2026. URL https://arxiv.org/abs/2602.10233.

Li-Hao Liu, Peng Tang, Kunwar Yashraj Singh, and Shabnam Ghadar. Espo: Error-structured prompt optimization via diagnose, diversify, and stabilize. arXiv (Cornell University), 2026. URL https://www.semanticscholar.org/paper/28eda5f75fa63a19fbd36d6540065ca806c0609a.

Elias Lumer, Sahil Sen, Vamse Kumar Subbiah, and Kevin Paul. Reflective prompt optimization of a

deep-research agent: A topic-disjoint, multi-seed protocol for measuring transferable gains and prompt-level overfitting. arXiv preprint, 2026. URL http://arxiv.org/abs/2610.12601v1.

Ruotian Ma, Xiaolei Wang, Xin Zhou, Jian Li, Nan Du, Tao Gui, Qi Zhang, and Xuanjing Huang. Are large language models good prompt optimizers? arXiv preprint, 2024. URL http://arxiv.org/abs/2402.02101v1.

Alexander Novikov, Ngân Vũ, Marvin Eisenberger, Emilien Dupont, Po-Sen Huang, Adam Zsolt Wagner, S. Shirobokov, Borislav M. Kozlovskii, Francisco J. R. Ruiz, Abbas Mehrabian, Manish Kumar, and Abigail See. Alphaevolve: A coding agent for scientific and algorithmic discovery. arXiv (Cornell University), 2025. URL http://arxiv.org/abs/2506.13131.

Tal Oved, Roi Pony, Oshri Naparstek, and Udi Barzelay. Evolution or illusion? rethinking evaluation in llm evolutionary search. arXiv (Cornell University), 2026. URL https://arxiv.org/abs/2609.19799.

Reid Pryzant, Dan Iter, Jerry Zheng Li, Yin Tat Lee, Chenguang Zhu, and Michael Zeng. Automatic prompt optimization with "gradient descent" and beam search. *Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 2023. doi: 10.18653/v1/2023.emnlp-main.494. URL https://doi.org/10.18653/v1/2023.emnlp-main.494.

Yewen Pu, Zachery Miranda, Armando Solar-Lezama, and Leslie Pack Kaelbling. Selecting representative examples for program synthesis. arXiv preprint, 2017. URL http://arxiv.org/abs/1711.03243v3.

Bernardino Romera-Paredes, Mohammadamin Barekatain, Alexander Novikov, Matej Balog, Manish Kumar, Emilien Dupont, Francisco J. R. Ruiz, Jordan S. Ellenberg, Pengming Wang, Omar Fawzi, Pushmeet Kohli, and Alhussein Fawzi. Mathematical discoveries from program search with large language models. *Nature*, 2023. doi: 10.1038/s41586-023-06924-6. URL https://doi.org/10.1038/s41586-023-06924-6.

Jingyuan Selena She, Christopher Potts, Samuel R. Bowman, and Atticus Geiger. Scone: Benchmarking negation reasoning in language models with fine-tuning and in-context learning. *Annual Meeting of the Association for Computational Linguistics (ACL)*, 2023. doi: 10.18653/v1/2023.acl-short.154. URL https://doi.org/10.18653/v1/2023.acl-short.154.

Agamdeep Singh, Srishti Gautam, Priyanshu Gupta, Nikita Mehrotra, Tanmay Bakshi, and Sumit Gulwani. Coding agents are strong prompt optimizers. arXiv (Cornell University), 2026. URL https://arxiv.org/abs/2609.26261.

Mirac Suzgun, Nathan Scales, Nathanael Schärli, Sebastian Gehrmann, Yi Tay, Hyung Won Chung, Aakanksha Chowdhery, Quoc V. Le, Ed H. Chi, Denny Zhou, and Jason Wei. Challenging big-bench tasks and whether chain-of-thought can solve them. arXiv preprint, 2022. URL http://arxiv.org/abs/2210.09261v1.

Chengrun Yang, Xuezhi Wang, Yifeng Lu, Hanxiao Liu, Quoc Viet Le, Denny Zhou, and Xinyun Chen. Large language models as optimizers. arXiv (Cornell University), 2023. URL http://arxiv.org/abs/2309.03409.

Mert Yuksekgonul, Federico Bianchi, Joseph Boen, Sheng Liu, Zhi Huang, Carlos Guestrin, and James Zou. Textgrad: Automatic "differentiation" via text. arXiv preprint, 2024. URL http://arxiv.org/abs/2406.07496v1.

Xiaofan Zhou and Lu Cheng. Explore more, validate less: Efficient reflective prompt optimization. arXiv preprint, 2026. URL http://arxiv.org/abs/2610.27367v2.